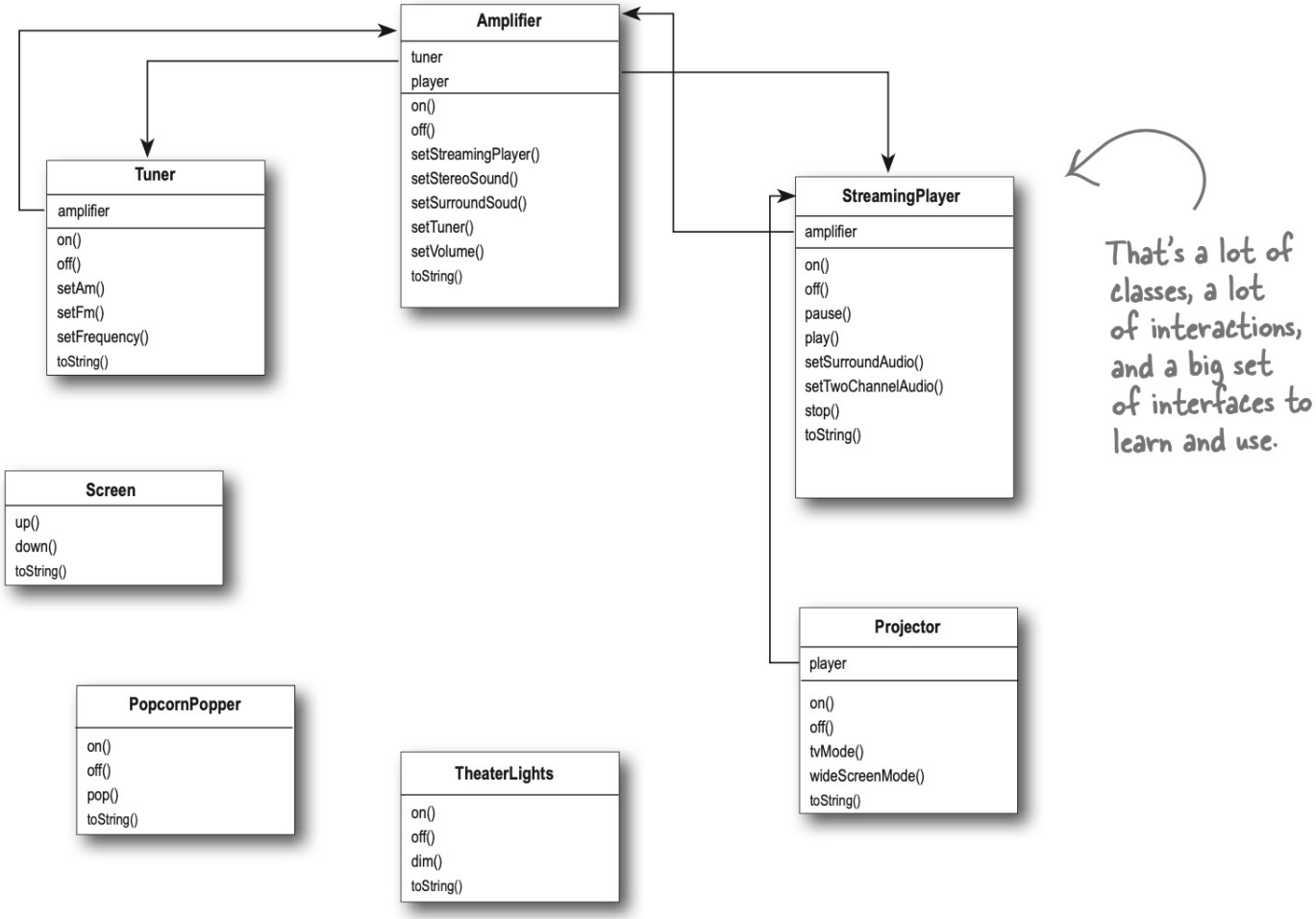
Padrões GoF – Fachada

Sabrina de F. Souto.

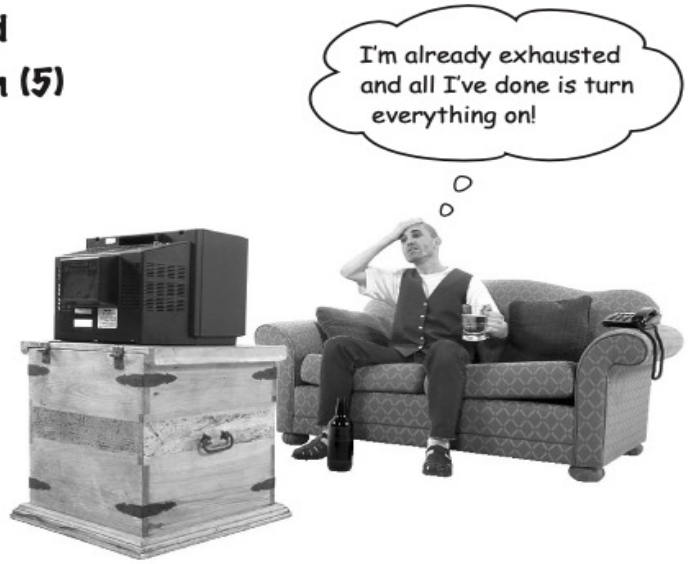
Home Theater



Fonte: Head First Design Patterns Building Extensible and Maintainable Object-Oriented Software

Assistindo a um filme da forma mais difícil

1. Turn on the popcorn popper
2. Start the popper popping
3. Dim the lights
4. Put the screen down
5. Turn the projector on
6. Set the projector input to streaming player
7. Put the projector on widescreen mode
8. Turn the sound amplifier on
9. Set the amplifier to streaming player input
10. Set the amplifier to surround sound
11. Set the amplifier volume to medium (5)
12. Turn the streaming player on
13. Start playing the movie



Assistindo a um filme da forma mais difícil

```
popper.on();
popper.pop();

lights.dim(10);

screen.down();

projector.on();
projector.setInput(player);
projector.wideScreenMode();

amp.on();
amp.setStreamingPlayer(player);
amp.setSurroundSound();
amp.setVolume(5);

player.on();
player.play(movie);
```

Turn on the popcorn popper and start popping...

Dim the lights to 10%...

Put the screen down...

Turn on the projector and put it in widescreen mode for the movie...

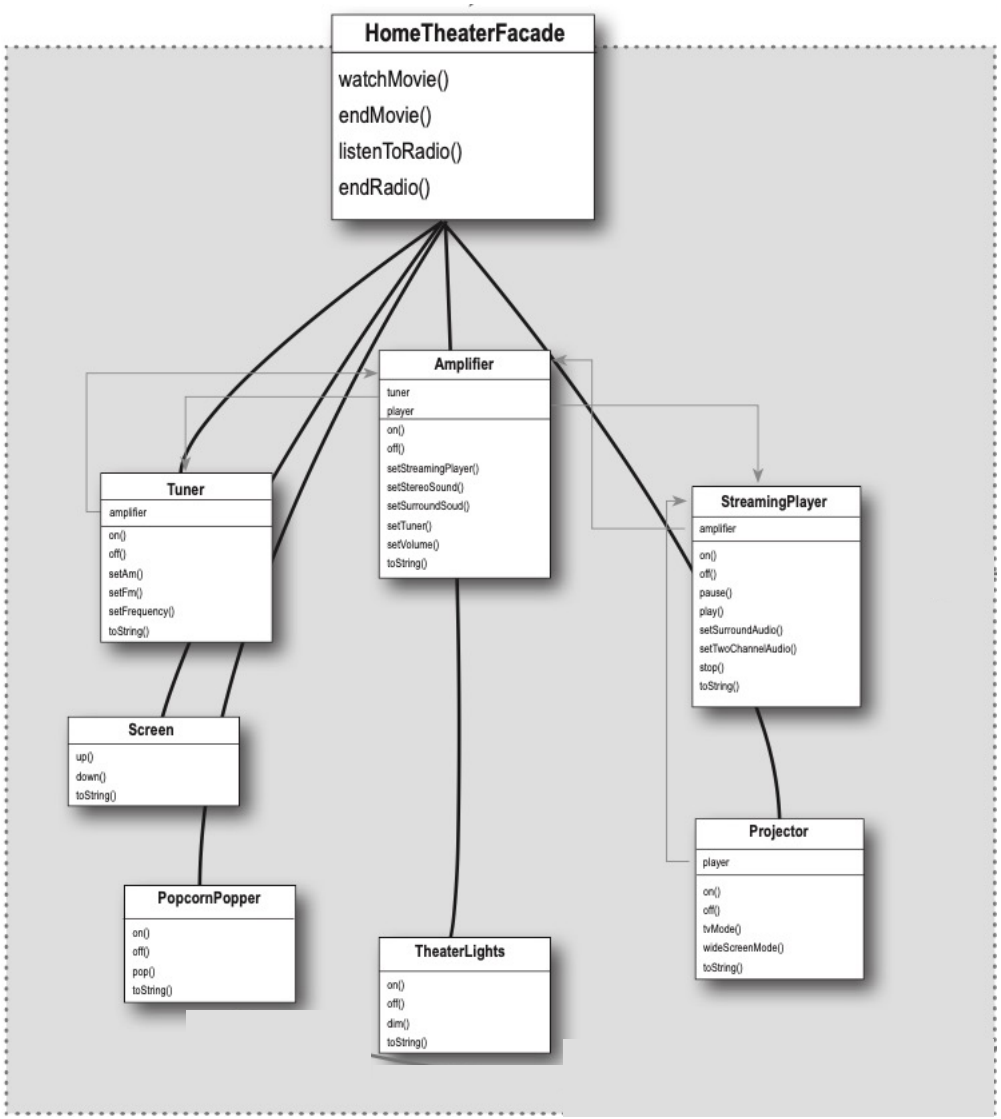
Turn on the amp, set it to Streaming player, put it in surround-sound mode, and set the volume to 5...

Turn on the Streaming player... and FINALLY, play the movie!

Six different classes involved!

- Ainda tem mais...
  - Quando o filme terminar, como você desligará tudo?
  - E, para ouvir CD ou rádio? O processo será igualmente complexo?
  - Se você decidir atualizar seu sistema?

Fachada para o seu Home Theater



Assistindo a um filme da maneira fácil

```
public class HomeTheaterTestDrive {
    public static void main(String[] args) {
        // instantiate components here

        HomeTheaterFacade homeTheater =
                new HomeTheaterFacade(amp, tuner, player,
                        projector, screen, lights, popper);

        homeTheater.watchMovie("Raiders of the Lost Ark");
        homeTheater.endMovie();
    }
```

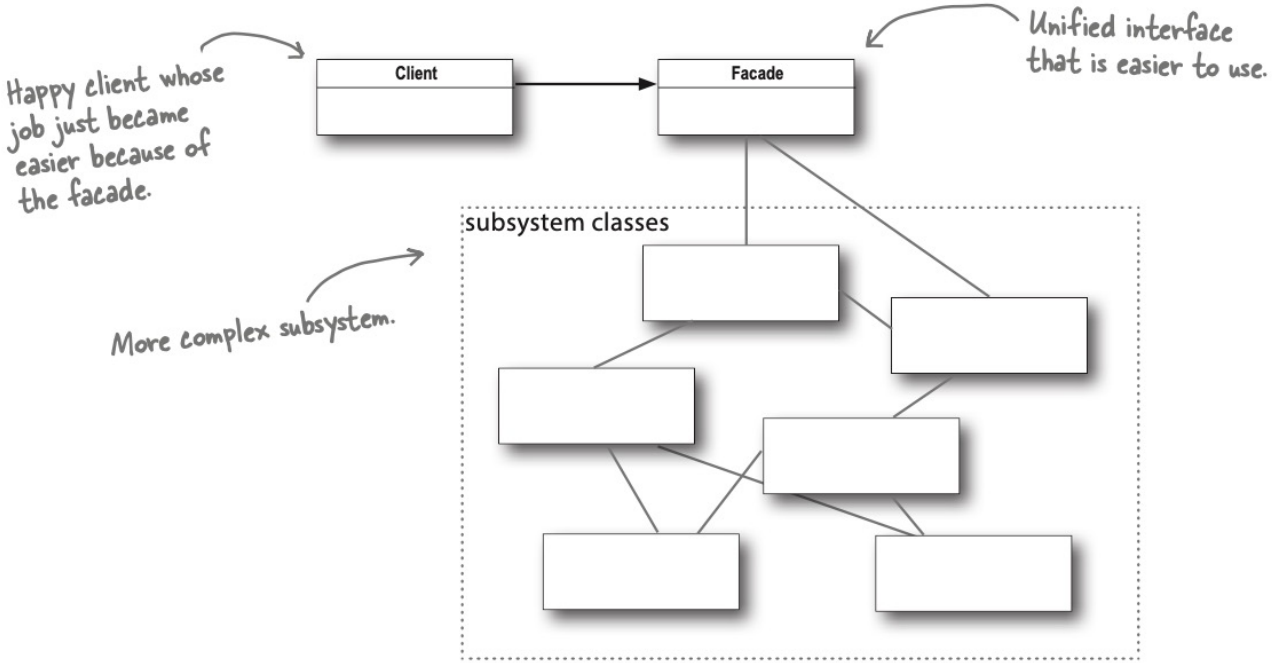
Here we're creating the components right in the test drive. Normally the client is given a facade; it doesn't have to construct one itself.

First you instantiate the Facade with all the components in the subsystem.

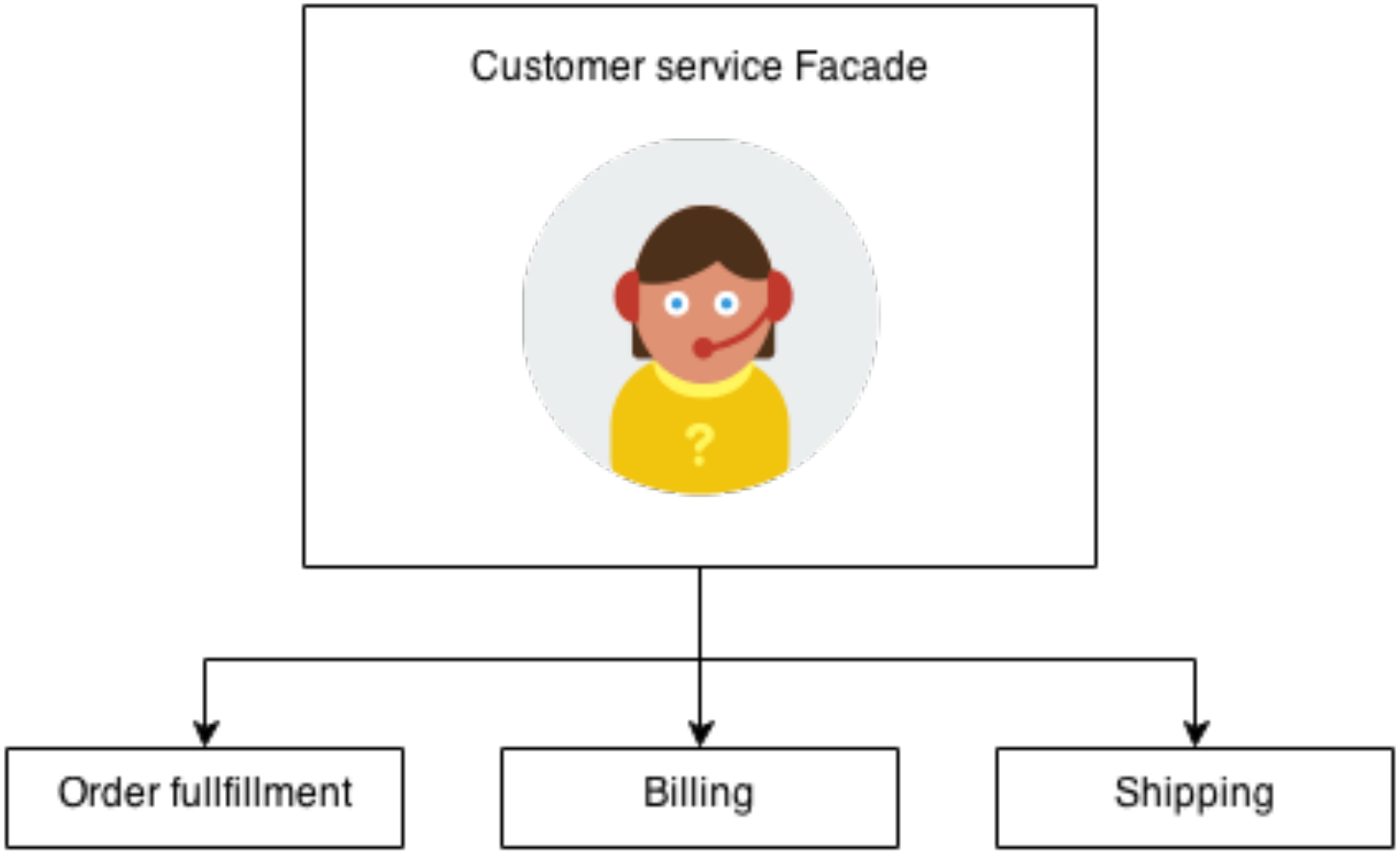
Use the simplified interface to first start the movie up, and then shut it down.

Fachada (ou Facade)

- Fornece uma **interface unificada** para um conjunto de interfaces de um **subsistema**.
- Define uma interface de nível mais alto que torna subsistemas mais fácéis de serem usados pelas classes cliente
  - O que são classes clientes?



Fachada (ou Facade)



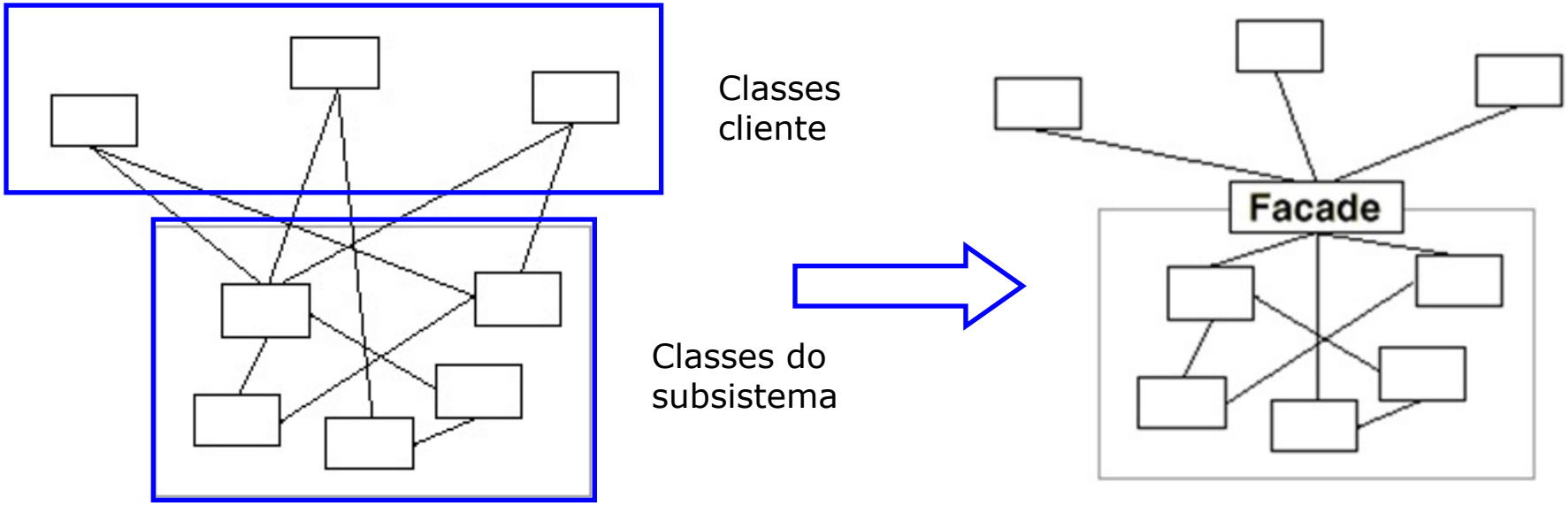
Fonte: https://sourcemaking.com/design_patterns/facade

Motivação

- O objetivo deste padrão é **diminuir o acoplamento e a dependência** entre classes

- É usado quando se deseja uma interface **simples** para um subsistema **complexo** ou para separar o subsistema em camadas

Motivação



- A Fachada fornece uma interface única e simplificada para os recursos e facilidades mais gerais de um subsistema

- Ponto único de acesso a todas as funções dos subsistemas.

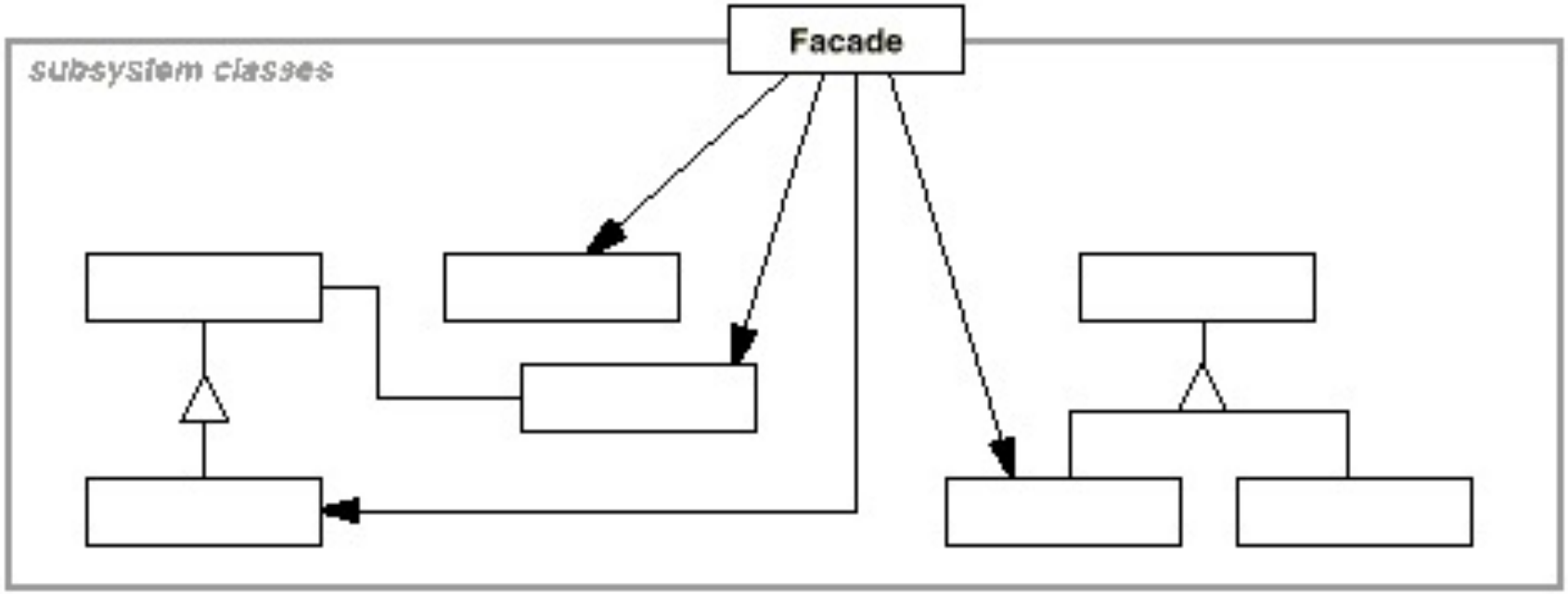
Motivação

- A Fachada fornece:
  - uma visão simples do sistema
    - omite o que não é necessário
  - independência e portabilidade dos subsistemas
    - Posso trocar meus subsistemas sem que tenhamos impacto nas classes clientes
    - Basta manter a interface de comunicação igual, ou seja, não mexer na assinatura dos métodos da fachada.

Quando usar?

- Deseja-se prover uma interface simples para um subsistema complexo
- Existem muitas dependencias entre clientes e as classes de subsistemas
- Deseja-se separar o subsistema em camadas

Estrutura



Participantes

- **Fachada**
  - sabe quais classes do subsistema são responsáveis por uma solicitação
  - delega solicitações do cliente aos objetos apropriados do subsistema

- **Classes do Subsistema**
  - implementam funcionalidade de subsistema
  - resolvem tarefas atribuídas pelo objeto *Facade*
  - não tem conhecimento da *Facade*, ou seja, não mantém referencias para ele

Colaborações

- Clientes se comunicam com o sistema ao mandar mensagens (métodos) para a Fachada que por sua vez passa aos objetos do subsistema.

- Clientes que usam a Fachada não precisam acessar os objetos do subsistema diretamente, o Fachada possui esses objetos.

Consequências

- **Subsistema mais fácil de se usar**:
  - Escode os componentes do subsistema dos clientes, reduzindo o número de objetos que os clientes lidam

- **Enfraquece o acoplamento** entre o subsistema e seus clientes.
  - Pode eliminar dependencias complexas ou circulares

- Não impede que as aplicações usem as classes do subsistema, se precisarem.

Dúvida?

Laboratório: Fachada

1. Implemente o sistema de informação geral (SIG) utilizado na UEPB. Suponha que este compreenda vários outros subsistemas responsáveis por atividades relacionadas a setores específicos.
   A partir do SIG é possível obter informações:
   - Administrativas (reuniões agendadas com a diretoria, entrevistas);
   - Financeiras (balanço de contas, folha de pagamento);
   - Relacionadas ao professores (alocação por disciplina, tempo de casa);
   - Relacionada aos alunos (histórico e RDM);
   - Ao almoxarifado (estoque, pedido de compra);
   - À infra-estrutura (alocação de salas).

Sugestões pré-laboratório:

- Faça um esbolso do diagrama UML
  - Identifique sistemas e fachada.

Itens avaliados:

- 20% - Implementação das classes e métodos;
- 40% - Uso adequado do padrão Fachada;
- 10% - Implementação da classe main como cliente da fachada;
- 30% - Testes.